The Bitstream Bandits

Bao Nguyen, Hans Yang

Today’s review

Established foundation

03 A reproducible baseline

04 Recovering the circuit

05 Two LLM workflows

06 What the hardware reveals

Next phase

07 Trojan shortlist and taxonomy

08 Guardrail work

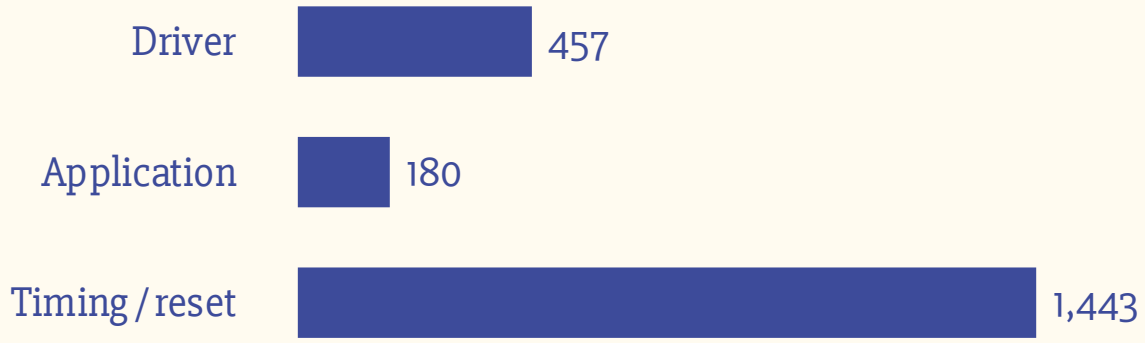
09 Division of labour

A reproducible baseline

Recovered hardware, reusable tests and frozen reference evidence

Inputs preserved	Circuit recovered	SPI driver tested	DUT timing tested	Baseline frozen
Bitstream + app	Mapped Verilog	Mock + scoreboard	Reset + readout	4856f24

Recorded clock pulses



Test scope	Checks
Driver + mock	75
Supplied application	8
Timing + reset	105

77 regression tests passed

Recorded integration run

Frozen reference: baseline-recovered-v1

Fault injection checks failure detection

Recovering the circuit

The recovered artifacts preserve a traceable reconstruction path



IceStorm: decode configuration and reconstruct logic

Yosys: process and export

Recorded Yosys sequence

Load iCE40 primitive models

Read the pin-mapped Verilog

Check hierarchy with top “recovered”

Lower processes and write simulation Verilog

Integration checks

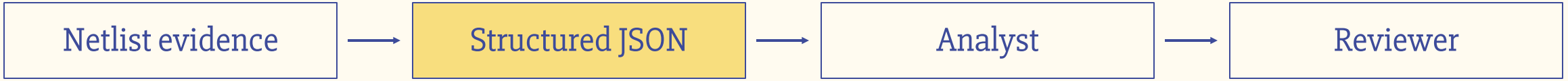
CS and mode pin mapping

Reset and output timing

Supplied application values

Two LLM workflows

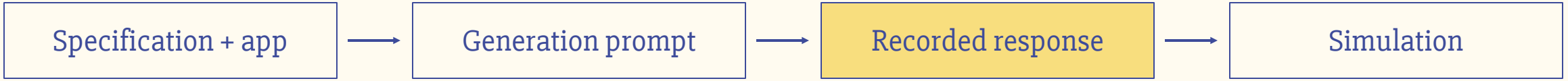
Recovery analysis



Pins, ROMs and signal traces

GPT-4o-mini through Portkey

Testbench generation



One SCK owner, timeouts, X/Z failures and fixed seeds

Saved generation runs replay Codex responses

7 structural checks passed; AI-generated interpretations remain unverified

What the hardware reveals

71
flip-flops

4
initialized ROM blocks

32-bit
MSB-first SPI

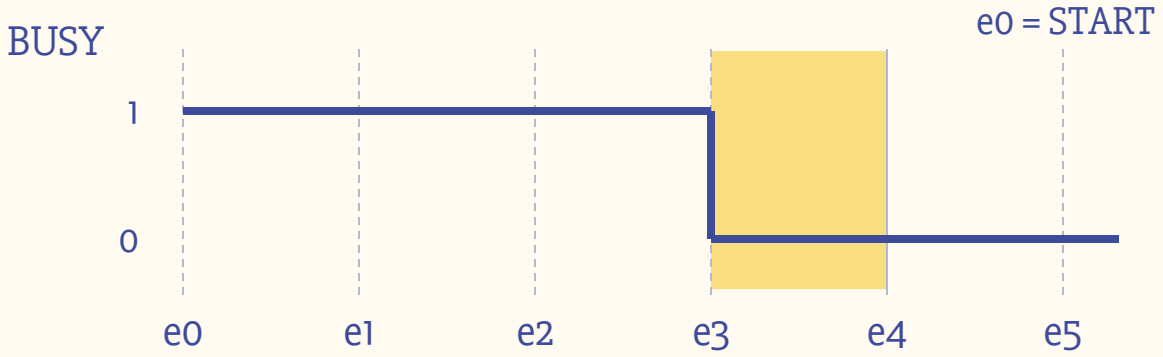
Supplied examples (hex)

Operation	Input	Observed
Echo	59c359c3	59c359c3
Encrypt	59c359c3	9cd84392
Decrypt	9cd84392	59c359c3

Clock reset and keep SCK running

Sample MISO 1 ns after rising SCK

BUSY falls before readout succeeds



After e3: old input still reads back

After e4: earliest tested successful read start

Cipher identity, key and arbitrary-input correctness remain unproven

Trojan candidates and taxonomy

Proposed shortlist:

Candidate	Trigger concept	Payload
Secret-key exfiltration	Two-word sequence	Expose key in output
Key-dependent fault oracle	Magic word + index	Key-dependent error
Encryption weakening	Rare input + mode	Bypass transformation
Ciphertext corruption	Specific input word	Flip an output bit
BUSY timing leakage	Operation sequence	Encode key in latency
Ciphertext replay	Rare operation sequence	Return earlier output
Mode hijack	Magic word + mode	Run the wrong mode
Selective denial of service	Word or sequence	Hold BUSY asserted

Design-stage change

Trigger

Combinational condition
Sequential activation

Payload target

Confidentiality / integrity
Availability

Key access is unproven

Not Yet

Division of labour

Hans leads recovery and design. Bao leads verification and automation.

Stage	Hans	Bao
Setup	IceStorm and Yosys	Icarus, GTKWave and AI logging
Initial work	Recover netlist and map FPGA I/O	Build SPI tests from docs and MicroPython
Understanding	Trace datapath and control with AI	Validate known input/output pairs
Trojan planning	Propose modification, trigger and payload	Assess testability, stealth and behavior
Implementation	Direct AI and integrate Trojan RTL	Direct AI for trigger and normal tests
Verification	Review simulations and debug RTL with AI	Simulate, synthesize and compare overhead
Deliverables	Recovery method, design and taxonomy	Tests, logs, automation and reproducibility

Thank You